后训练学习计划 • 第 13 周（11/30-12/6）

本周定位：**Phase 4 第一周——面试火力准备启动**。从这周起**不再训模型**：任务是 LeetCode Hot 100 启动（每天 3 道新题 + 1 道复习，周一到周六各再扛 1 道"补名额"新题——**本周 ≥25 道的差额就是这么来的**）、手撕 ML 每周 3 题、八股第一批（Transformer 与训练技巧）、**简历终版**、**投递启动**（三档目标清单 + 内推渠道 + 台账）、第 1 次模拟面试。 性质变了，节奏也要跟着变。前 12 周的成就感来自"曲线动了、模型能跑了"，接下来 4 周的成就感只来自"AC 了、投出去了、约到面了"。**这三件事的反馈更慢更冷，需要制度而不是热情来撑**——所以本周最重要的产出不是题目数，是那套台账和判据。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗红字 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	制度建立 + 数组/哈希启动	`jobhunt/` 四文件 + 台账模板 + 4 新 1 复 AC	6h
Day 2 周二	双指针/滑动窗口 + 手撕 attention 变体	4 新 1 复 + `ml_attention.py` 单测绿 + 八股 12 条	6.5h
Day 3 周三	子串/普通数组 + 手撕 DPO loss	4 新 1 复 + 自写 DPO loss 与 TRL 对拍通过	6h
Day 4 周四	链表（上）+ 手撕 BCE/CE + **简历终版**	4 新 1 复 + 对拍绿 + 简历 v2 每数字有出处	6.5h
Day 5 周五	链表（下）+ 三档目标清单 + 首批投递	4 新 1 复 + 清单 ≥40 家 + 已投 ≥8 家	6h
Day 6 周六	混合复习 + **第 1 次模拟面试** + 周复盘	4 新 1 复 + 模拟面记录 + 复盘 ≥10 条	6.5h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + 周 14 预习	2 新 2 复 + 验收 ≥6/7 项通过	4h

注：**本周卡时上限 4h，默认 0h**。唯一允许开卡的理由是"简历/叙事里的某个数字需要补跑证据"（例：`teach200` 的 judge 一致性百分比当时只写在终端里没存表）。**规则：为补证据开卡前，先在 `COST.md` 写清要跑什么、预计几小时、这个数字进简历哪一条**；写不出对应简历条目就不许开。超过 4h → 那个数字从简历上删掉（降级方案，成本为零）。上周遗留的"加 replay 重跑"这类**改善型**实验，本周一律不做，排到周 14 之后或直接放弃。

注：**本周最大风险不是题难，是三件事同时拖**：（1）刷题"看过"当成"会了"（判据里堵死）；（2）简历数字与博客/报告不一致（周 12 的溯源表今天要用）；（3）投递一直没启动，第四周还在"再准备一下"（所以**首批 ≥8 家排在 Day 5，不许往后挪**）。**砍单顺序**：Day 4 手撕 BCE/CE（挪到周 14 的手撕名额）→ Day 6 的模拟面试改"对镜自述录屏"（**不许取消，只许换形式**）→ Day 2/3 的"补名额"新题 → 其余不许砍，**投递和台账永不砍**。

注：关于落差，一句实话：**前 12 周建立的技术资产不会因为这周不训练而消失，但会因为说不清楚而贬值**。所以本周所有练习（八股、手撕、模拟面）都按"能开口讲 + 能手写"两条线做，不按"看懂"做。刷完一道题的标准动作是：合上编辑器，对着空气 60 秒讲清思路与复杂度。讲不清 = 没做。

二、开始前：目录 / 台账 / 判据制度（约 50 分钟，Day 1 任务 1 就地做完）

1. **本地目录**（bash 全用正斜杠）：

```
mkdir -p E:/study/posttrain/jobhunt/{leetcode,mlcoding,bagu,resume,mock}
mkdir -p E:/study/posttrain/week13/{day1,day2,day3,day4,day5,day6}
cd E:/study/posttrain/llm-posttraining-lab && git tag m4-final && git push --tags
```

2. **把 lab 仓库设为"只读 + 补文档"**：从本周起，`llm-posttraining-lab` 里只允许改 README、补 `NOTICE-DIFFERENCES.md`、修文档链接，**不改训练脚本、不改超参**。打了 `m4-final` 这个 tag，之后所有对外链接都指向它（免得一周后你自己都对不上哪个版本是最终版）。
3. **台账文件**：`E:/study/posttrain/jobhunt/applications.csv`，表头写死：

```
公司,档位,岗位名,渠道,内推人/内推码,投递日,当前状态,最后动作日,复盘链接
```

4. **刷题记录**：`leetcode/log.md`，一行一题：`日期 | 题号 | 题名 | 结果(AC/未AC) | 提交次数 | 用时 | 卡在哪 | 复习日`。
5. **手撕 ML 记录**：`mlcoding/log.md`，一行一次：`日期 | 题目 | 限时 | 是否独立写完 | 对拍是否通过`。

完成判据：三个目录组、两个 log 文件、一个台账 csv 全部建好；`m4-final` tag 推上去（`git ls-remote --tags` 能看到）。**台账表头少一列都算没完成**——少的那一列两周后你一定想查。

> 注：`applications.csv` 用逗号分隔但字段里会出现逗号（岗位名"算法实习生（后端训练方向）"没有逗号，但渠道"BOSS直聘/牛客"用斜杠），**统一：字段内禁用逗号，用斜杠**。这条现在不看，等你写第 15 行时表就废了。

Day 1 • 周一：制度建立 + 数组/哈希启动

今日总览（6h）：今天把接下来 4 周的**制度**立起来（判据、台账、复盘模板），并把 Hot 100 的第一组（数组/哈希）跑起来。今天开始，"这道题我会"这句话不再算数，只有 AC 截图算数。卡时 0。

任务 1：判据制度 + 复盘模板（约 50 分钟）

1. 在 `jobhunt/rules.md` 写死四条（**本周及以后所有判据的地基**）：
   - **新题判据**：从读题到通过（Accepted）**不许打开题解、不许搜题**、不许问 AI；允许只看题目本身与我以前的 `log.md` 笔记。
   - **提交次数 ≤2**：第 3 次提交前必须先在纸上重写思路；靠反复试错蹭过的 AC 记 `AC-fragile`，进复习队列优先。
   - **超时处置**：单题 **45 分钟**未 AC → 停，写"我卡在哪一步"，看题解**只看到思路为止**（不许看代码），关掉，**第二天重做**；只有第二天独立 AC 才计入本周题量。
   - **复习规则**：每题 AC 后 24h、72h 各重做一次（记 `复习日`）；复习题不计入新题数，但**复习不通过要把该题重新排队**。
2. 建 `jobhunt/interview-review.md` 模板（周 16 会沿用同一张表，现在就固定下来）：

```
## 复盘 · <日期> · <公司/模拟面>
| 题/问 | 我答了什么 | 正确答案/更优答法 | 下次怎么说（口播版 ≤80 字） |
|---|---|---|---|
```

完成判据：`rules.md` 四条齐；模板表头齐且下面留 ≥10 空行；`log.md` 里已有 `结果` 列可选值 `AC / AC-fragile / 未AC`（三个值就够，别发明更多状态）。

> 注：为什么"看题解后第二天重做才计数"——因为**面试官现场给的就是你没见过的题**，而你在他眼里只有当下这一次表现。计数口径松一寸，面试时的真实水平就差一截。这一条如果你嫌烦想跳过，说明你还没进入求职周。

任务 2：OJ 环境与提交习惯（约 35 分钟）

1. 决定主战场（二选一，**不要两个都用**）：LeetCode 中国站（题多、面经在牛客有对应帖）或 牛客网（面试环境、限时赛、题解区能看别人怎么讲）。
2. 统一本地调试方式（推荐，省掉在网页里 debug 的时间）：

```
mkdir -p E:/study/posttrain/jobhunt/leetcode/py
```

```
# two_sum.py —— 本地用示例跑一遍再提交，避免语法错浪费一次提交
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        ...

if __name__ == "__main__":
    print(Solution().twoSum([2, 7, 11, 15], 9))   # -> [0, 1]
```

3. 提交前固定三查：函数签名对不对、返回类型对不对（`List[int]` vs `int`）、**边界**（空数组 / 单元素 / 全相同 / 负数 / 溢出）。

完成判据：`two_sum.py` 本地打印 `[0, 1]` 后提交 AC；今天起每次提交前口头说出"三查"（做不动就写张纸条贴屏幕边上，一周后就不用了）。

任务 3：刷题 Day 1 —— 数组/哈希 4 新 + 1 复（约 235 分钟）

按这个顺序做（**每题限时 45 分钟，复习题 55 分钟**）：

序	题号	题名	你要练到的点
新 1	1	两数之和	哈希表一次遍历；能说出为什么不能用双指针（无序）
新 2	49	字母异位词分组	键的设计（排序串 vs 计数元组）与两种复杂度对比

序	题号	题名	你要练到的点
新3	242	有效的字母异位词	26 计数数组；追问点：字符集是 Unicode 怎么办、文件超大怎么办
新4	128	最长连续序列	只用 set、`while` 只在"链头"启动 → 能讲清为什么是 O(n)
复1	—	从 `log.md` 里挑一道 72h 到期的旧题（今天没有就挑一道你当年写过但现在想不起来的简单题）	重做到一次 AC

完成判据：**4 道新题全部独立 AC**（不看题解、提交 ≤2 次），台账每题一行、"卡在哪"不许写"没卡"以外的空值；`128` 那道能在 60 秒内口播 O(n) 的原因。**当天 AC 数 <4 → 不熬夜追，记进 `log.md` 并明天排在最前面。**

> 注：`128` 是 Hot 100 里最容易被"看答案觉得很简单"的题。判据不只是 AC：如果你写的是"排序后遍历"，那是 O(n log n)，面试会当场让你优化——**今天必须再写一遍 set 版**，两个版本都留下，这题就是本周第一份"我比题解多走一步"的素材。

任务 4：八股第一批开工（约 30 分钟）

1. 建 `bagu/transformer.md`，把周 3/5 的笔记重排成**问答格式**（问一句 + 口播答 ≤80 字），今天只写 8 条：`MHA 的形状流转`、`为什么要 /sqrt(d)`、`causal mask 加在哪一步之前`、`KV cache 为什么省算力`、`attention 复杂度为什么是 O(n^2)`、`softmax 注意力里 score 的数值范围与溢出`、`multi-head 里每个头到底在学什么（能不能说"子空间"）`、`残差连接与初始化为什么让深层可训`。
2. 每条下面再加一行「**追问**：面试官下一句最可能问什么」。

完成判据：8 条问答 + 每条有追问行；口播一遍总时长 ≤4 分钟（掐表）。

> 注：八股不要"读"，要"说 + 写"。**判断标准是嘴不是眼**：同一道题，闭眼口播和手写各一遍，手写的漏洞通常比口播多，因为面试手撕环节考的是手写。周 14 会把这批八股改成"限时默写"，现在先把口播底稿存下来。

任务 5：收工笔记（约 10 分钟）

睡前 3 分钟在 `NOTES.md` 记一行：今天最卡的一道题 + 明天第一件事。

完成判据：`NOTES.md` 有今天的日期开头一行，且包含**具体题号**（写"那道Hard的"不算）。

今日验收：`rules.md` 四条 + 台账模板齐 + **4 新 1 复 AC** + 八股 8 条。

Day 2 • 周二：双指针/滑动窗口 + 手撕 attention 变体

今日总览（6.5h）：刷题进双指针与滑动窗口；手撕 ML 第 1 题（attention 变体，正好是周 1-3 的东西，今天起它从"我会"变成"我 20 分钟能默写出来"）。卡时 0。

任务 1：刷题 Day 2 —— 4 新 + 1 复（约 235 分钟）

序	题号	题名	要点
新 1	283	移动零	双指针原地写，一次遍历
新 2	11	盛最多水的容器	为什么移动较矮的一侧（这一步的论证要能讲）
新 3	15	三数之和	排序 + 双指针 + **去重三处**（最容易错的地方）
新 4	3	无重复字符的最长子串	滑窗模板成型：右扩、左缩条件、答案更新时机
复 1	128	最长连续序列（72h 到期重做）	只看题目，一次 AC

完成判据：4 新独立 AC；`15` 的三处去重全部说清位置；`3` 的窗口模板抄成 `leetcode/template_sliding_window.py`，之后所有滑窗题都用它起手。

> 注：`11` 和 `15` 都是"贪心式移动"的题，面试官爱追问**证明**。不必写严格证明，但要能说出反证的一句话："如果固定较矮的边、移动较高的边，宽度必减且高度不会超过原来的较矮边，所以面积不可能变大。"这一句能背下来，比刷十道同类题有用。

任务 2：手撕 ML #1 —— attention 变体（MHA / MQA / GQA + KV cache）（约 85 分钟）

新建 `mlcoding/attention_variants.py`，**不看任何资料**（周 1/3 你写过 `attention.py` 和 `block.py`，今天是从零默写）：

1. `scaled_dot_product_attention(Q, K, V, causal=True)`：形状 `(B, T, C)` 输入，含 heads 切分与拼接。
2. `mqa_attention(...)`：K/V 只有 1 个头，Q 有 h 个（靠 `expand` 广播）。
3. `gqa_attention(..., n_kv_groups)`：h 个 query 头共享 `h / n_kv_groups` 组 KV。
4. `kv_cache_step(...)`：增量解码一步——把新 token 的 k/v `cat` 进缓存再算注意力；**必须证明它与一次性全序列前向一致**。
5. 单测：`test_attention_variants.py`，用 `torch.manual_seed(0)` 与 **float64**，三条断言：形状对、上三角权重全 0、`max(new_kv_out, full_out) 差 < 1e-9`。

```
source E:/study/posttrain/.venv/Scripts/activate
cd E:/study/posttrain/jobhunt/mlcoding && python -m pytest -q test_attention_variants.py
```

完成判据：**pytest 4 passed**（三种变体的形状 + causal + 与一次性前向对拍 + MQA/GQA 在 `n_kv=1` 时退化为同结果的断言）；口播 60 秒讲清"GQA 省的是什么、为什么不省精度"（KV cache 显存 ÷ 组数，Q 的表达力没动）。

> 注：本地 Windows 上 `import torch` 报 `OMP: Error #15 Incident detected`（conda 的 MKL 与 torch 自带 OpenMP 打架）——正解就是用这个独立 venv 而不是 conda base；急着往下走就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）或 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过。**对拍一定统一 float64**：bf16/fp32 下 1e-9 的容差毫无意义，你会发现"差一点就通过"然后开始怀疑人生。

任务 3：八股 Transformer 第一批（约 60 分钟）

`bagu/transformer.md` 追加 12 条并复测前 8 条：`RoPE 的原理与相对位置性质`、`为什么长上下文要插值/外推`、`RMSNorm vs LayerNorm`、`Pre-LN 为什么比 Post-LN 稳`、`FFN 为什么是 4C`、`SwiGLU 与 GLU`、`权重共享（weight tying）`、`温度与 top-k 对输出的影响`。每条给「问 / 80 字答 / 追问」。

完成判据：文件累计 **≥20 条**；把前 8 条遮住答案自测，**≥6 条能完整口播**；不行的标 `[重]` 明天再测。

任务 4：收工笔记（约 10 分钟）

`mlcoding/log.md` 记手撕 #1 结果；`NOTES.md` 一行。

完成判据：`log.md` 里今天这条包含**卡在哪一行**（例："GQA 的 reshape 顺序反了，注意力权重对不上"），不许只写"完成了"；`NOTES.md` 有日期开头的两行（Day 1 / Day 2）。

今日验收：4 新 AC + pytest 4 passed + 八股 ≥20 条 + 自测 ≥6/8。

Day 3 • 周三：子串/普通数组 + 手撕 DPO loss

今日总览（6h）：刷题进"前缀和 + 滑窗"与经典数组题；手撕第 2 题是 DPO loss——**周 8 手推过、周 11 跑过，今天要能 20 分钟手写出来并与 TRL 对拍**。这是后训练面试的手撕必考题。

任务 1：刷题 Day 3 —— 4 新 + 1 复（约 235 分钟）

序	题号	题名	要点
新 1	560	和为 K 的子数组	前缀和 + 哈希计数；`cnt[0]=1` 为什么必须初始化
新 2	76	最小覆盖子串	滑窗"收缩型"模板，`need/window/valid` 三件套
新 3	53	最大子数组和	Kadane；追问：要返回区间下标怎么写
新 4	56	合并区间	排序键的选择 + 遍历合并；边界 `[1,4],[2,3]`
复 1	3 或 15	昨天到期的一道	一次 AC

完成判据：4 新独立 AC；`560` 能说清"为什么不能用双指针"（有负数，窗口单调性不成立）；`76` 用今天的滑窗模板重写而不是另起炉灶。

> 注：`560` 是"看似滑窗其实必须前缀和"的经典陷阱，也是面试官最爱用来区分"背过模板"和"真懂单调性"的题。**把"能不能滑窗"的判据背下来**：窗口扩大时目标量是否单调，且左边界移动是否只会让答案变好。两个条件任一不成立就不能滑窗。这句话在周 14 复盘时还要用一次。

任务 2：手撕 ML #2 —— DPO loss（约 70 分钟）

新建 `mlcoding/dpo_loss.py`：

1. 手写 `dpo_loss(policy_chosen_logps, policy_rejected_logps, ref_chosen_logps, ref_rejected_logps, beta=0.1)`，返回 `(loss, rewards_chosen, rewards_rejected)`；**用 `-F.logsigmoid` 而不是 `torch.log(torch.sigmoid(x))`**（数值稳定，这是必考点）。
2. 梯度 sanity：全 0 的 logits 差 → loss 应等于 `ln 2 ≈ 0.6931`。
3. **与 TRL 对拍**（这是今天的硬判据）：造一个 4 条偏好对的小 batch，用同一份 `ref` logprob，分别用你的函数与 TRL `DPOTrainer` 的 `odds_ratio_loss`（或当前版本的对应内部函数）算，**绝对差 < 1e-4**。

```
import torch, torch.nn.functional as F

def dpo_loss(pc, pr, rc, rr, beta=0.1):
    logs = beta * ((pc - rc) - (pr - rr))
    loss = -F.logsigmoid(logs).mean()
    return loss, (beta * (pc - rc)).detach(), (beta * (pr - rr)).detach()

t = torch.tensor(0.0)
assert abs(dpo_loss(t, t, t, t, 0.1)[0].item() - 0.6931471805599453) < 1e-6
print("DPO SANITY OK")
```

完成判据：打印 `DPO SANITY OK`；对拍脚本打印 `DPO CROSSCHECK OK (diff=<一个具体数>)`，**diff < 1e-4**；能默写 `L_DPO` 的完整式子（含 `π_θ` 与 `π_ref` 四项）。

> 注：对拍时最容易对不上的三处：① TRL 默认按 **token 级 logprob 求和**（不是平均），你要保证自己也是求和；② `loss_type` 默认 `sigmoid`（标准 DPO），别配成 `ipo/orpo`；③ mask：`rejected` 里 padding 的位置没屏蔽。**diff 卡在 1e-2 级别基本都是这三条之一**，别去怀疑公式。

任务 3：八股第一批（训练技巧）（约 45 分钟）

`bagu/training.md` 写 10 条：`AdamW 与 Adam 的区别（权重衰减解耦）`、`warmup 与 cosine 各自解决什么`、`最大学习率探针（binary search LR）`、`bf16 vs fp16 + GradScaler，为什么 4090 上优先 bf16`、`梯度累积的等效 batch 与 BN 的坑`、`clip_grad_norm_ 是 global norm（不是逐参数）`、`gradient checkpointing 用时间换什么`、`显存构成四项（权重/梯度/优化器态/激活）`、`ZeRO 三个阶段各切什么`、`DDP 与 FSDP 的通信差别（讲概念）`。

完成判据：10 条齐 + **其中 3 条带你自己的一手数字**（引周 4 的显存/速度对照表、周 5 的发散排查、周 11 的 KL 曲线），出处写文件路径。

> 注：八股分两种：背得出的和用过一次的。**带自己数字的那三条就是你的护城河**，面试官问"梯度裁剪"时你答的是"我周 4 实测 global norm 从 3.2 裁到 1.0，step/s 掉了 8% 但 loss 曲线不再抖"——这一下就和背题的人分开了。今天先做这件事：把前 12 周里所有实测数字过一遍，能挂到八股上的全挂上。

任务 4：收工笔记（约 10 分钟）

`mlcoding/log.md` 记手撕 #2 的对拍结果（**把 diff 的最大数量级抄进去**，例 `max |Δ|=3.1e-12`）；`NOTES.md` 一行。

完成判据：`log.md` 里那一行有**具体 diff 数**和一句"下次手撕 DPO 要先写的两行是什么"；`NOTES.md` 有今天日期开头的一行。

今日验收：4 新 AC + `DPO CROSSCHECK OK` + `training.md` 10 条（3 条带自己数字）。

Day 4 • 周四：链表（上）+ 手撕 BCE/CE + 简历终版

今日总览（6.5h）：上午刷题进链表（Hot 100 链表占约 14 道，本周先啃前 4 道，周 14 继续）；下午把**简历终版**做完——这是本周最重要的单件交付物，**必须做到每个数字都有文件出处**。

任务 1：刷题 Day 4 —— 链表 4 新 + 1 复（约 220 分钟）

序	题号	题名	要点
新 1	206	反转链表	迭代 + 递归两种都写；递归的"返回后指针怎么接"讲清
新 2	141	环形链表	快慢指针；为什么必相遇（一步一句证明）
新 3	142	环形链表 II	相遇后求入环点的数学；**写在纸上再敲代码**
新 4	160	相交链表	双指针换头的 O(1) 空间写法
复 1	560	前缀和重做	一次 AC

完成判据：4 新独立 AC；**链表题一律用"画图 + 三行伪代码"起步**（画的照片存 `leetcode/img/`，≥4 张）；`142` 的推导能在纸上写清（`a + c + k(a+b) = n(a+b)` 那一步）。

> 注：链表是手撕最容易翻车的地方——**指针改了顺序导致断链**是第一大错。今天开始固定习惯：改指针前先把要用的 next 存到临时变量，写完再动原指针；单测里必查"长度没丢、无环、头尾对"。周 14 还要刷 8 道链表，今天这 4 道是它的地基。

任务 2：手撕 ML #3 —— BCE / CE 的数值稳定实现（约 60 分钟）

新建 `mlcoding/losses.py`：

1. `my_bce_with_logits(logits, targets)`（用 `softplus - z*y` 形式，不许先 sigmoid 再 log）与 `my_cross_entropy(logits, y)`（减 max 后 logsumexp）。
2. 对拍：`torch.nn.BCEWithLogitsLoss` / `F.cross_entropy`，**float64、`torch.manual_seed(0)`、shape (1024, 32)、含极端 logits（±50）**，绝对差 < 1e-10。
3. 反传对拍：`logits.requires_grad_(True)`，两种实现的 `.grad` 差 < 1e-8。

完成判据：打印 `BCE OK` / `CE OK` / `GRAD OK` 三行；口播 60 秒"为什么 SFT/DPO/RL 的损失全是 CE 家族，以及为什么必须合并 softmax 与 log"。

> 注：为什么专门考这个——`log(0)` 与 `exp` **上溢是训练里最常见的 NaN 来源之一**，而大多数候选人只会写 `torch.log(softmax(x))`。周 1 的 CE 梯度对拍、周 5 的发散排查、周 11 的 NaN 诊断，本质上都是这 20 行的事；面试手撕把它写对，胜过讲十分钟理论。

任务 3：简历终版（约 95 分钟，今天的正事）

在 Phase 0 的简历 v0 上出 **v2 = 终版**（一页，PDF 命名 `简历-姓名-大模型后训练实习-v2.pdf`）。项目经历四条（M1-M4）各 3-4 个 bullet，**每条统一句式：动词 + 技术 + 数字 + 结果**。

写法对照（左不要用 / 右要这样）：

不要用	要这样（数字全部替换成你自己的实测值）
"负责模型微调与优化"	"在 Qwen2.5-1.5B 上以 LoRA(r=16, α=32) 做 SFT（ 条指令数据），held-out JSON 合法率 %→%"
"使用 DPO 提升生成质量"	"构造 对偏好数据（强模型打分 + 100 条人工复核，一致率 %），DPO(β=0.1, lr=5e-6) 使盲评胜率 %"
"做了 GRPO 实验"	"实现三项可验证奖励（格式/CEFR/事实锚），GRPO(G=4) reward 均值 **→，并定位到组内方差塌缩的失效场景"**
"参与评测体系建设"	"自建 200 题教学评测集（五类配额、13-gram 去污染 0 命中），LLM-as-judge 固定 rubric 与 temperature=0，60 样本重跑 3 次一致率 %"
"部署模型节省成本"	"vLLM 起服务 + 开关式灰度，30 个真实场景 A/B：单条成本 ¥ vs ¥**（未计入运维与迭代成本），p95 延迟 ms vs ms"**

每条后面挂一个 `（源：<文件路径>）`（源只写在自己的 `resume/sources.md` 里，不写进简历正文）。

完成判据： 1. **每个数字都能在** `resume/sources.md` **里查到文件与列名**，抽查 5 个数字当场打开源文件对上（对不上就把这个数字删掉，不许"差不多"）； 2. 一页、无形容词（"深入/丰富/大量/优秀"一个不许留）； 3. 找一位没跟过你这 12 周的人看 60 秒，让他复述你做了什么——**复述不出"M4 是什么"就重写第一条**； 4. 三档岗位版本各一份差异说明（业务侧版把 A/B 与成本放最前，模型公司版把 GRPO 与奖励设计放最前）。

> 注：**简历诚信是硬线**。你 16 周里做的东西真，但简历天生有夸大空间。三条禁令：不写"负责"（你是唯一执行者，就用"实现/训练/评测"）；不写"线上大规模 A/B"（你的 A/B 是 30 个真实场景 + 影子流量，**不是**面向真实用户的灰度——这句话周 12 已经写过一次，今天再核一次）；不引用别人的指标当自己的结果。被戳穿一次，前面所有真东西一起作废。

任务 4：八股复测 + 收工笔记（约 15 分钟）

遮住答案复测 `transformer.md` 里带 `[重]` 的条目，全部清掉为准； `mlcoding/log.md` 记手撕 #3 的三个 diff 数量级。

完成判据： `[重]` 标记**清零**（grep 该文件无 `[重]`）； `log.md` 今天的三条手撕记录各带一个 diff 数。

今日验收：4 新 AC + 三个 `OK` 打印齐 + **简历 v2 定稿且 5 个抽查数字对得上** + `[重]` 清零。

Day 5 • 周五：链表（下）+ 三档目标清单 + 首批投递

今日总览（6h）：今天把投递这台机器启动起来。**投递不许"等我再准备两天"**——周 12 的验收已经把技术资产封版了，现在缺的就是反馈周期，越早投越早知道差什么。

任务 1：刷题 Day 5 —— 4 新 + 1 复（约 225 分钟）

序	题号	题名	要点
新 1	21	合并两个有序链表	哨兵节点写法；递归版一句话复杂度
新 2	2	两数相加	进位处理；不许转成整数（会溢出，这是追问点）
新 3	19	删除链表的倒数第 N 个结点	快慢指针间距；**虚拟头节点**处理删头
新 4	876	链表的中间结点	快慢指针；偶数长度返回哪一个（说清题意）
复 1	142	入环点重做	一次 AC

完成判据：4 新独立 AC；链表累计独立 AC ≥8 道；`log.md` 里今天四题都有"卡在哪"的具体描述（例："删头没加虚拟节点，提交 2 次才过"）。

> 注：**投递日还刷 4 道题，是不是本末倒置？** 不是，这恰好是设计：投递的反馈周期是 3-10 天，等待期就是刷题期，两件事不抢时间。反过来，如果你从今天开始"专心等回复、改简历第 8 版"，两周后你会既没有面试也没有手感。

任务 2：三档目标清单（约 60 分钟）

建 `jobhunt/targets.md`，三档各 ≥13 家（合计 ≥40 家，周 15 集中投到 ≥20 家时才有的挑）：

档	例子（以当期在招 JD 为准）	定位
大厂实验室	字节 Seed、通义、腾讯混元、美团 LongCAT、快手 KlingLM	门槛最高，重训练与算法深度
模型公司	DeepSeek、智谱、Kimi（月之暗面）、MiniMax、阶跃、百川	后训练方向最明确，月之暗面/智谱/美团对无论文最友好
业务侧 AI 团队	教育（作业帮/猿辅导/字节教育）、出海、智能座舱（蔚来/理想）	**门槛更友好，最适合作为第一个实习跳板**；你的"产品 + 数据 + 评测"闭环在这里最保值

每家一行：`公司 | 档位 | 岗位名 | JD 关键词 | 我匹配哪三条 | 缺口哪两条`。

完成判据：≥40 行、每行"匹配三条 + 缺口两条"都写满；**把缺口重复出现的那 2-3 项标出来**（多半是"分布式训练"和"论文/竞赛"）——它们直接决定你周 14-16 的补强方向，也是总计划风险 3（横移投数据构造/评测/Agent 岗）的判断依据。

> 注：不要只挑"最想去"的 10 家写。投递是概率游戏，清单薄的直接后果是：三周后没回复就开始怀疑整个方向。**40 家是这个游戏能跑起来的最低样本量**。

任务 3：首批投递 ≥8 家 + 台账（约 65 分钟）

1. 渠道分工（写进 `targets.md` 末尾）：官网/公众号（正式通道，必须走）、BOSS 直聘与实习僧（反馈快）、牛客招聘 + 面经帖下常有 HR、**内推**：牛客/即刻/小红书搜员工内推码，或直接找在职学长学姐；内推码要记进台账（后续问进度用得着）。

2. 首批投 **≥8 家**（业务侧 ≥5 + 模型公司 ≥2 + 大厂实验室 ≥1，别一上来全投最难的），每家：简历 v2 对应版本 + 3 行自述（第一句就是你的 200 字叙事压缩版）。
3. 全部登记进 `applications.csv`（九个字段一个不许空，拿不到信息写 `N/A` 而不是留空）。

完成判据：`applications.csv` **≥8 行有投递日**；每家能说出"我投的是哪个岗、渠道是谁"；内推 ≥3 家（有内推码/内推人姓名）。

> 注：自述别写"我对大模型充满热情"。写："我有真实产品（含用户日志）→ 自己造数据 → SFT/DPO/GRPO 三阶段 → 自建 200 题评测 + 灰度 A/B → 换掉 API 的成本测算，全部代码在 。"这一句就是你的差异化，也是 HR 三十秒内判断要不要往下看的东西。

任务 4：收工 + 台账快照（约 10 分钟）

把 `applications.csv` 复制一份为 `week13/ledger_day5.csv` 存档，并 `grep -c "" applications.csv` 记下行数。

完成判据：快照文件存在且行数 = 台账行数 - 1（表头差），快照里**每一行的"状态"列都不是空值**；`log.md` 有今天四道题的"卡在哪"。

今日验收：4 新 AC + 清单 ≥40 家 + **投递 ≥8 家且台账齐** + 内推 ≥3 家。

Day 6 • 周六：混合复习 + 第 1 次模拟面试 + 周复盘

今日总览（6.5h）：上午把本周的题按"复习队列"清一遍并加做链表压轴；下午做**第 1 次模拟面试**（牛客互面 / 朋友 / 对镜录屏，三选一，**不许取消，只许换形式**），晚上当场复盘。第一次模拟面的目标不是表现好，是**把你会卡壳的地方全部暴露出来**。

任务 1：刷题 Day 6 —— 4 新 + 1 复（约 235 分钟）

序	题号	题名	要点
新 1	146	LRU 缓存	**手撕常客**：哈希 + 双向链表；先自己写结构再考虑 `OrderedDict`
新 2	234	回文链表	快慢指针 + 反转后半 + **还原现场**（追问必考）
新 3	20	有效的括号	栈；`{[()]}` 型最小模板
新 4	739	每日温度	单调栈：为什么栈里存下标、为什么每元素进出各一次
复 1	本周 `AC-fragile`	全部重做	一次 AC 才划掉

完成判据：4 新独立 AC；`146` **能在 20 分钟内从零写完**（这题是手撕题，不是刷题——写不快说明双向链表的指针功夫不够）；`AC-fragile` 队列清零。

> 注：`146` 与 `234` 属于"面试现场写"级别的题，判据要更严：**闭卷 + 限时 + 一次 AC**。`739` 是单调栈入门，周 14 还会遇到 harder 的（接雨水 `42` 就在周 14 的图/栈附近），今天先把"进出各一次所以 O(n)"这个论证说顺。

任务 2：第 1 次模拟面试（约 60 分钟，严格计时）

形式（三选一，优先级从高到低）：牛客找人互面（你面他一次 + 他面你一次）> 同学/朋友照题单面 > 对镜录屏自述（**必须有录像**）。

流程与计时：

段	时长	内容
自我介绍	2min	用周 12 的 200 字叙事，不许超时
项目深挖	20min	讲 M4（用 15 分钟讲稿的骨架），对方按 `FAILURES.md` 追问
八股	20min	Transformer 10 条 + 训练技巧 10 条随机抽 8 问
手撕	15min	attention 变体 或 DPO loss（二选一，让对方选，别自己挑熟的）
反问	3min	准备 3 条反问

完成判据：有录像或对方签字的题单；**全程没有一句"这个我大概知道"**（说了就记一条缺陷）。

任务 3：模拟面复盘（约 40 分钟）

填 `interview-review.md` 今日那一节。

完成判据：**≥10 行**（不足 10 行说明你没认真看录像），每行三列都填：我答了什么 / 正确答案或更优答法 / 下次怎么说的口播版（≤80 字）；末尾列出「下次面之前必须补的 3 件事」。

> 注：复盘的第三列（**下次怎么说**）是这张表的唯一价值。只写"这题答错了"等于没复盘——两周后你面对同一题会**一模一样地再答错一遍**，因为缺的是话术结构不是知识。把口播版念三遍，比再刷 5 道题更改变你的表现。

任务 4：周复盘 + gaps（约 55 分钟）

`review.md` 五块：① 硬产出（AC 数 / 手撕 / 简历 / 投递数 / 模拟面）；② 时间真实开销（对 41.5h，按天记）；③ 本周暴露的最大短板（一句话）；④ 卡时与花费（应为 0h / ¥0，除非为补证据开过卡——那要贴 `COST.md` 依据）；⑤ 周 14 最想改的一件事。

完成判据：五块齐 + `gaps.md` 更新（技术洞 → 周 14/15 具体条目；缺口 → 周 15 集中投递的策略调整）。

今日验收：4 新 AC + 模拟面有记录 + 复盘 ≥10 行 + review 五块。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + 周 14 预习

今日总览（4h）：求职周的周日不是休息日，是**台账核对日**：题目有没有欠账、投递状态要不要跟进、复盘有没有下次可用的话术。

任务 1：刷题 Day 7 —— 2 新 + 2 复（约 130 分钟）

序	题号	题名	说明
新 1	88	合并两个有序数组	从后往前填，边界很啰嗦但面试爱考
新 2	155	最小栈	设计题：辅助栈 vs 差值栈，两种都写
复 1	146	LRU 重做	限时 20 分钟，一次 AC
复 2	76	最小覆盖子串	一次 AC

完成判据：2 新 + 2 复全部独立 AC；**本周新题独立 AC 累计 ≥25**（去 `log.md` 里数，数不出来就是没达标）。

任务 2：补漏（约 40 分钟）

规则：最多补 2 个洞，贪多必失。优先：本周未 AC 被排进队列的题 → 模拟面复盘里"下次怎么说"还写不出来的条目。

完成判据：2 条闭合，各写一句"补法 + 证据（AC 链接或录像时间点）"。

任务 3：闭卷复测（约 25 分钟，别跳）

纸上写完三题，不查任何资料： 1. 手写 `my_cross_entropy`（减 max + logsumexp），并写出它对 logits 的梯度（应为 `p - onehot`）； 2. 写出 DPO 的完整损失与 `β` 的作用，回答"β 太大/太小分别怎样"； 3. 说出 GQA 相比 MQA 与 MHA 各自的取舍，以及"为什么 KV cache 显存能按组数除"。

完成判据：**三题全对**才算通过；任一条错，把它连同对应的八股条目一起写进周 14 的"限时默写"清单（周 14 的手撕题就是 LoRA 层、Adam、numpy 前向，这三题是它的前置）。

任务 4：本周总验收（约 25 分钟）

#	验收项	判据
1	刷题量	本周新题**独立 AC ≥25**、`AC-fragile` 清零、`log.md` 每题有"卡在哪"
2	复习制度	24h/72h 复习按时执行，复习队列无欠账
3	手撕 ML	3 题全绿：attention 变体（pytest 4 passed）、DPO loss（`DPO CROSSCHECK OK` diff<1e-4）、BCE/CE（三个 `OK`）
4	八股第一批	`transformer.md` + `training.md` 合计 ≥30 条，每条有追问行；抽 10 条口播 ≥8 通过
5	**简历终版**	v2 一页定稿；抽查 5 个数字全部对得上源文件；三档版本差异说明齐
6	**投递与台账**	目标清单 ≥40 家、`applications.csv` **本周投递数 ≥8**（内推 ≥3）、状态字段无空值
7	模拟面试	**面试数 ≥1**（有录像或题单）、**复盘条目数 ≥10**、"下次面之前必补 3 件事"明确

完成判据：**≥6 项通过 = 本周合格**；第 5、6 项为**强制项**（简历与投递没过，整周就是没过）。判定方式：每格现场打开文件/终端核对并填 `通过/未通过 + 证据路径`（题数看 `log.md` 的 AC 行数，投递数看 `applications.csv` 的投递日列，不许凭印象打勾）。**第 1 项不过怎么办**：不许周末通宵补，把缺口按题号写进周 14 Day 1 的前 2 小时，并检查是不是"45 分钟超时看题解"这一步你偷偷放宽了——九成原因是这个。

> 注：**求职周特别口径**：判据一律用「独立 AC（不看题解）」而不是「看过」；验收表里的「本周投递数 / 面试数 / 复盘条目数」三个数**每周必填**，这是 Phase 4 唯一的 KPI。技术能力已经封版了，接下来的 4 周只产这三类数字。

任务 5：周 14 预习（约 20 分钟）

周 14 的四件事先知道长什么样： 1. 刷题换到链表/二叉树/BFS-DFS/回溯，**累计 ≥55**（本周 25-26，下周净增约 30）； 2. 手撕换成 **LoRA 层、Adam、numpy 前向**——前两个你有周 3/4 的对拍底子，`numpy` 前向是最难的一个，先把 nanoGPT 的 `Block.forward` 形状表默写一遍当预热； 3. 八股第二批：RL 基础、reward hacking、评测方法、数据构造——**你的 `FAILURES.md`（周 11+12 合并那份 ≥10 条）就是这一批的主教材**，今天先通读一遍，把每条改写成「现象→定位→修复」的 80 字口播版（做 3 条即止，周 14 做完剩下的）； 4. 第 2 次模拟面。

完成判据：形状表默写一次（≥10 行 `(B,T,C)` 流转）+ `FAILURES.md` 口播版 3 条 + 预习四件事各自建好待办。

今日验收：总验收 ≥6 项（含两项强制项）+ 闭卷三题全对 + 预习齐。

三、常见问题与纪律

- **"投了没回复"怎么办？** 这是常态，不是信号。**唯一正确的反应是继续按日计划走**（刷题 + 八股 + 下一批投递），同时把每次被拒的岗位名与 JD 关键词记进 `targets.md`——连续三家因为同一个要求拒你，那才是信号（多半是分布式/论文，处理办法见总计划风险 3：横移投数据构造、评测、Agent 后训练岗，技能重叠约八成、竞争小得多）。**不许**因为没回复就停下来改简历第 8 版或重跑实验：那是最舒服的拖延方式。
- **某天没完成怎么办？** 按"天-任务号"记进 `gaps.md`，Day 7 就是缓冲。**永远优先保 Day 4 的简历终版和 Day 5 的首批投递**——它们有外部时间窗（提前批在 2027 年 2-3 月开抢，但日常实习现在就在招），而题目下周还能刷。
- **刷题要不要按标签刷完再换？** 不要。Hot 100 本身就是按标签分组的，照它走；但每天最后一题**从上一个标签的队列里抽**（本周的链表题里夹一道昨天的滑窗），交错练习才留得住。
- **卡时与花费**：本周上限 4h、**默认 0h**。你大概不会需要用卡。真要用：先写清跑什么、多少钱、进简历哪一条；超预算就把那个数字删掉。跑完立刻关机（**不是销毁**），在 `COST.md` 回填真实值。
- **手撕 ML 的判据为什么比刷题严**（要"对拍通过"）：因为面试官看的是**你对自己代码的信任来源**。写出来 + 对拍通过 + 说清数值容差为什么这么定，三件事一起做才叫会手撕。只写出来不算。
- **AI 工具的使用边界**（求职周版本）：允许——面经检索、把报错解释给你、模拟面时当面试官出题、review 你写完的简历话术。**不允许**——替你刷题（你 AC 的每题都要能讲）、替你写手撕实现（对拍脚本可以问，实现不行）、替你回答"你项目难点是什么"（那是你最不该假手于人的 60 秒）。
- **节奏**：周一到周六每天 6-6.5h、周日 4h。**固定"上午刷题、下午八股与求职动作"的顺序**——刷体力题在前是因为它要求清醒，八股与投递允许在状态差时做。这个顺序一旦定下来，四周都别改。

- **最后一句**：你的技术故事比大多数同期候选人完整，短板在"手速 + 表达 + 运气"。手速和表达这四周能练，运气靠投递量。**别把运气的问题留给自我怀疑去解决。**